Chat with Energy Documents (RAG)

Course Name: GenAI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	**Mahak Shrimal**	**EN22CS301566**
02	**Mahak Taose**	**EN22CS301567**
03	**Mohit Patel**	**EN22CS301610**
04	**Naina Manghani**	**EN22CS301626**
05	**Paavan Patni**	**EN22CS301671**

Group Name: 02D6

Project Number:GAI-13

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-2026

1. INTRODUCTION

1.1. SCOPE OF THE DOCUMENT

This document defines the low-level design for the "Chat with Energy Documents" system based on Retrieval-Augmented Generation (RAG). The system enables users to upload energy-related documents such as PDFs and interact with them through a question-answering interface. It covers internal processing including document ingestion, chunking, embedding generation, vector storage, and query retrieval.

1. Defines internal workflow of RAG pipeline
2. Describes interaction between components (LLM, Vector DB, UI)
3. Explains data handling and query processing
4. Focuses on implementation using Python, Streamlit, FAISS, and OpenAI APIs

1.2. INTENDED AUDIENCE

This document is intended for stakeholders involved in development and evaluation of the system.

1. **Developers**-To understand code structure and implementation
2. **AI/ML Engineers**-To review embedding and retrieval logic
3. **Project Evaluators**-To assess correctness and completeness
4. **Students**-For academic submission and viva explanation

1.3. SYSTEM OVERVIEW

The system is a RAG-based intelligent document assistant that processes uploaded PDFs and enables context-aware Q&A. It uses a pipeline where documents are converted into embeddings and stored in a vector database. When a user asks a question, relevant chunks are retrieved and passed to the LLM to generate accurate answers.

1. Upload PDF → Extract Text
2. Split into chunks
3. Convert into embeddings
4. Store in FAISS vector database
5. Retrieve relevant chunks during query
6. Generate answer using LLM

2. SYSTEM DESIGN

2.1 APPLICATION DESIGN

The application follows a modular pipeline architecture integrating UI, preprocessing, vector storage, and LLM response generation.

1. **Frontend Layer (Streamlit)-**Handles file upload and user queries
2. **Processing Layer-**Text extraction using PyPDFLoader.Chunking using RecursiveCharacterTextSplitter
3. **Embedding Layer-**OpenAI Embeddings convert text → vectors
4. **Storage Layer-**FAISS stores vector embeddings
5. **Retrieval + LLM Layer-**RetrievalQA fetches relevant chunks.ChatOpenAI generates final response

2.2. PROCESS FLOW

The process starts with document upload and ends with answer generation.

1. User uploads PDF via Streamlit
2. File is stored temporarily
3. PDF is loaded and converted into text
4. Text is split into chunks (size=500, overlap=50)
5. Each chunk is converted into embeddings
6. Embeddings are stored in FAISS
7. User enters query
8. Top relevant chunks (k=3) are retrieved
9. LLM generates context-based answer

2.3 INFORMATION FLOW

Information flows sequentially through different components of the RAG pipeline.

1. Input → PDF Document
2. Processing → Chunked Text
3. Embedding → Vector Representation
4. Storage → FAISS Index
5. Query → Retriever
6. Output → LLM Generated Answer

2.4 COMPONENTS DESIGN

1. **Streamlit UI-**Handles user interaction (upload + query)
2. **PyPDFLoader-**Extracts text from uploaded PDFs
3. **Text Splitter-**Divides text into manageable chunks
4. **OpenAI Embeddings-**Converts text into vector embeddings
5. **FAISS Vector DB-**Stores and retrieves embeddings efficiently
6. **RetrievalQA Chain-**Combines retrieval + LLM response generation

2.5 KEY DESIGN CONSIDERATIONS

1. **Accuracy-**Responses are grounded in uploaded documents only
2. **Scalability-**FAISS allows handling large document sets
3. **Efficiency-**Chunking reduces processing complexity
4. **Modularity-**Each component (loader, splitter, DB, LLM) is independent

2.6 API CATALOGUE

1. **OpenAI Embedding API-**Used for generating vector embeddings
2. **OpenAI Chat API (GPT-3.5)-**Used for answer generation
3. **FAISS API-**Used for similarity search and retrieval

3. DATA DESIGN

3.1 DATA MODEL

The system uses a document-centric data model.

1. **Document-**Contains raw PDF text
2. **Chunks-**Smaller segments of document
3. **Embeddings-**Vector representation of chunks
4. **Query-**User input question
5. **Response-**Generated answer from LLM

3.2 DATA ACCESS MECHANISM

1. FAISS provides fast similarity search
2. Retrieval is based on cosine similarity
3. Top-k results (k=3) are fetched

3.3 DATA RETENTION POLICIES

1. Vector embeddings stored locally
2. Temporary files used for PDF processing
3. No long-term storage of user queries

3.4 DATA MIGRATION

1. FAISS index can be saved and reloaded
2. Can be migrated to cloud vector DB (Pinecone, Weaviate)

4. INTERFACES

1. User Interface-Streamlit web interface
2. Backend Interface-Python-based RAG pipeline
3. External APIs-OpenAI APIs for embeddings and LLM

5. STATE AND SESSION MANAGEMENT

1. FAISS index maintains vector state
2. Session handled by Streamlit
3. No persistent user sessions

6. CACHING

1. Embeddings stored locally in FAISS
2. Avoids recomputation for same documents
3. Improves response time

7. FUNCTIONAL REQUIREMENTS

7.1 DOCUMENT UPLOAD

1. User can upload PDF documents
2. System validates file format

7.2 DOCUMENT PROCESSING

1. Extract text from PDF
2. Split into chunks

7.3 EMBEDDING GENERATION

1. Convert text into vector embeddings
2. Store in FAISS

8. NON-FUNCTIONAL REQUIREMENTS

8.1 SECURITY

1. API keys should be secured
2. File handling should be safe

8.2 PERFORMANCE

1. Fast retrieval using FAISS
2. Efficient chunking improves speed

8.3 SCALABILITY

1. Can handle large documents
2. Can integrate cloud vector DB

9. REFERENCES

[1] LangChain Documentation, "LangChain Official Docs," 2025. [Online]. Available: https://python.langchain.com/

[2] OpenAI, "OpenAI API Documentation," 2025. [Online]. Available: https://platform.openai.com/docs/

[3] Facebook AI Research, "FAISS: A library for efficient similarity search," 2025. [Online]. Available: https://github.com/facebookresearch/faiss

[4] Streamlit Inc., "Streamlit Documentation," 2025. [Online]. Available: https://docs.streamlit.io/

[5] PyPDF Documentation, "pypdf Library Documentation," 2025. [Online]. Available: https://pypdf.readthedocs.io/